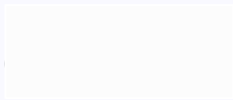


Projet : Plateforme d'exploration de graphes en C++

Jean & Arthus

2025-2026 · 4IMI



Plan

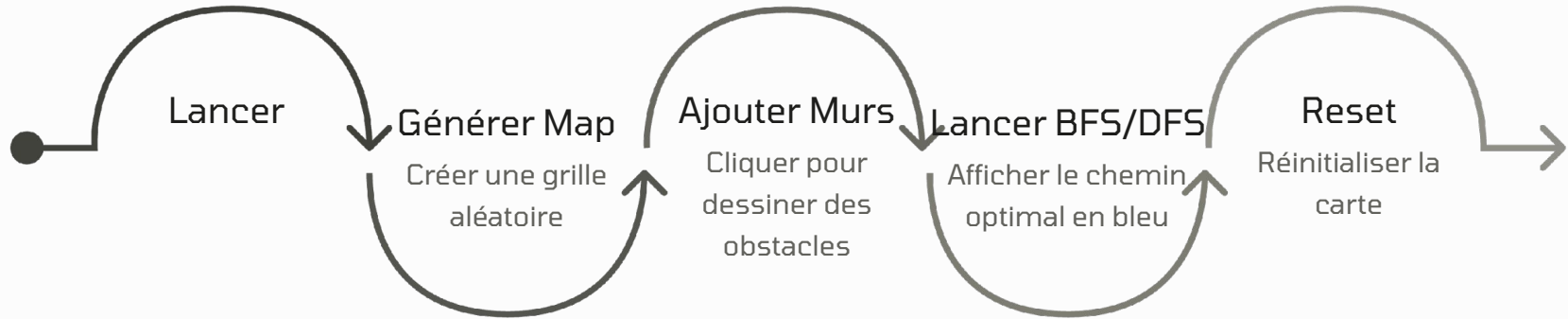
- **1. Contexte et objectif du projet**
- **2. Démonstration**
- **3. Architecture du Projet (MVC)**
- **4. Modèle de données**
 - a. Case
 - b. Map
- **5. Partie Logique :**
 - a. Algorithme DFS
 - b. Algorithme BFS
 - c. Autres
- **6. Interface Qt affichage**
- **7. Les limites et améliorations possible**

1 - Contexte et consignes

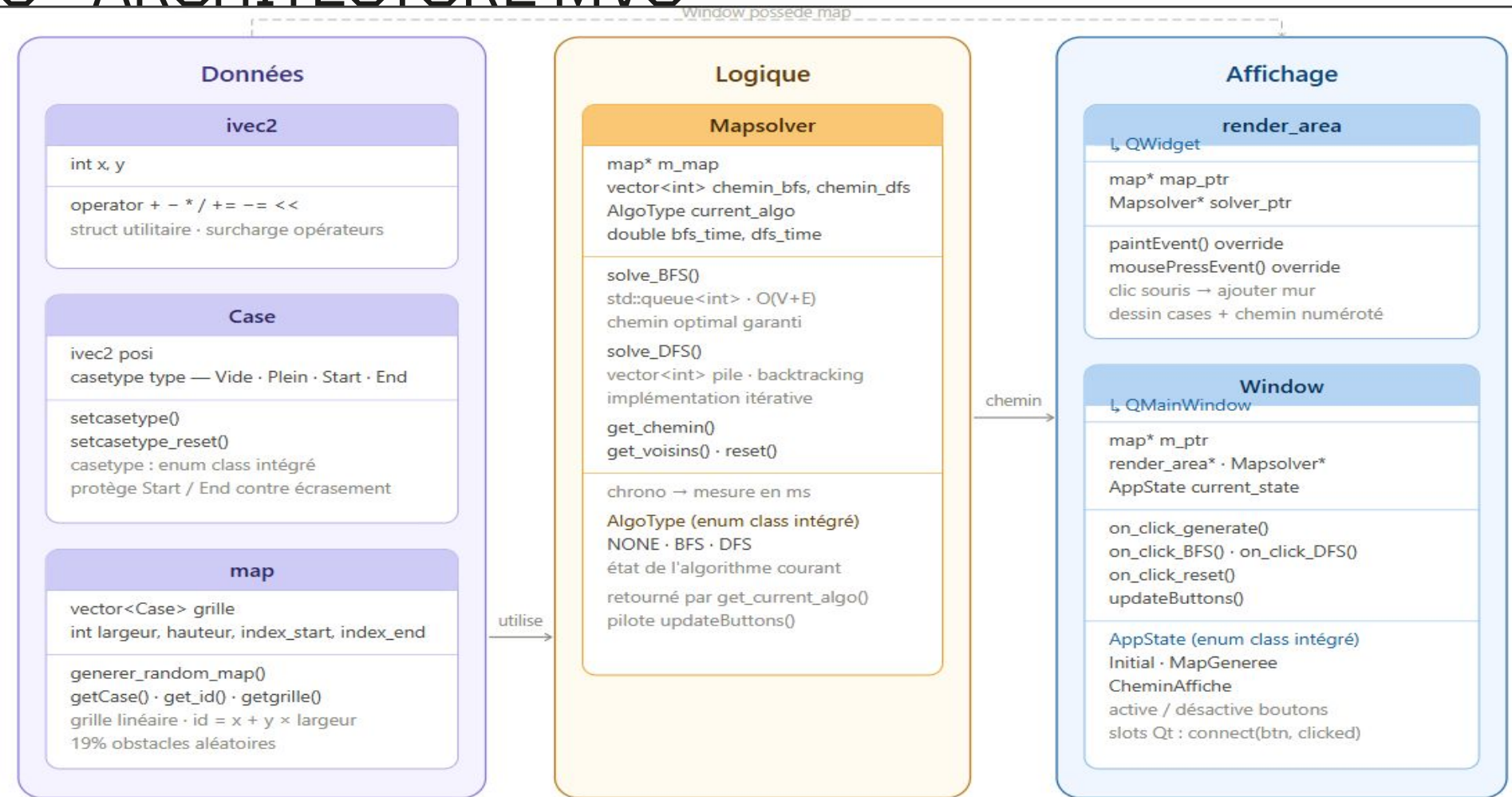
L'objectif de ce projet est de concevoir et développer une plateforme interactive permettant de démontrer et comparer différents algorithmes d'exploration de graphes.

- *Développement du projet en trois axes :*
 - Modèle de graphe et logique (validation en terminal)
 - Interface graphique avec Qt (visualisation en temps réel)
 - Algorithmes d'exploration et outils d'analyse

2- Démonstration



3 - ARCHITECTURE MVC



4 - Modèle de données

la classe case

```
enum class casetype{Vide, Plein, Start, End};
```

```
class Case{
```

```
public :
```

```
    Case(ivec2 posi, casetype type);
```

```
    ivec2 getcoord() const;
```

```
    casetype getcasetype() const;
```

```
    void setcasetype(casetype type);
```

```
    void setcasetype_reset(casetype type);
```

```
private :
```

```
    ivec2 posi;
```

```
    casetype type;
```

la classe case

private:

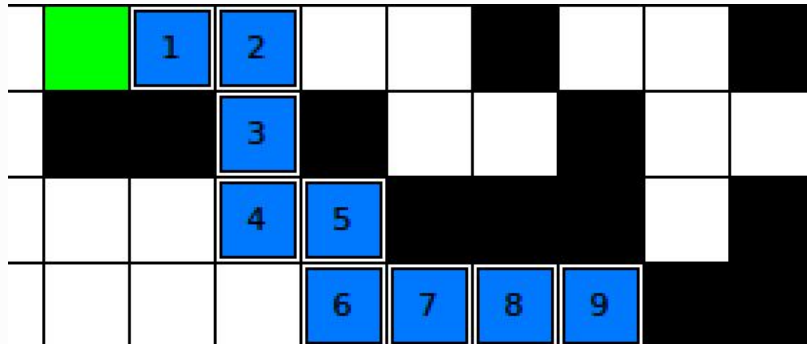
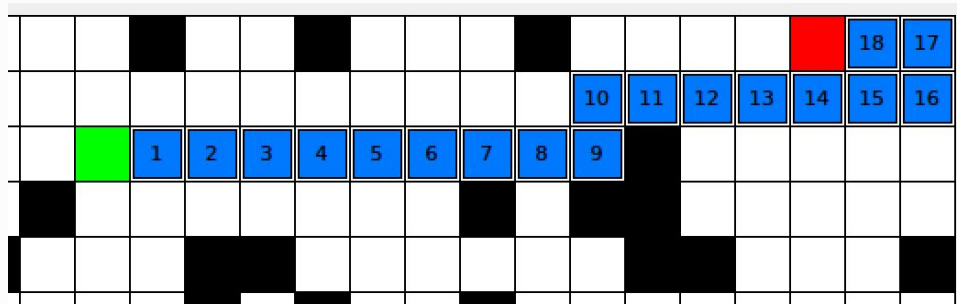
```
std::vector<Case> grille;  
int largeur;  
int hauteur;  
int cell_size;  
int index_start = -1;  
int index_end = -1;
```

public:

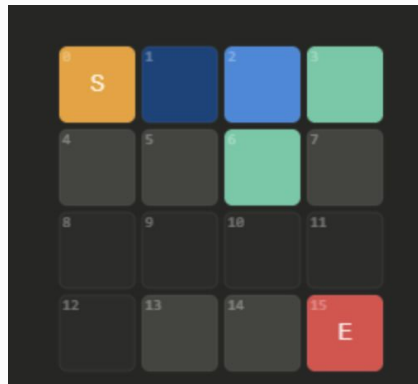
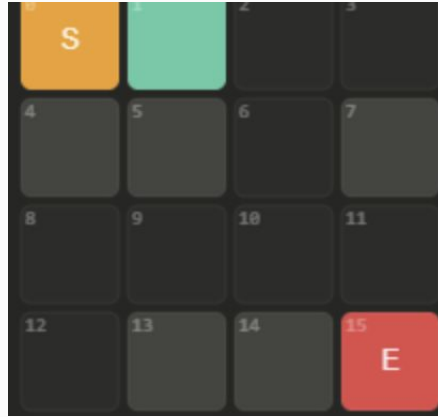
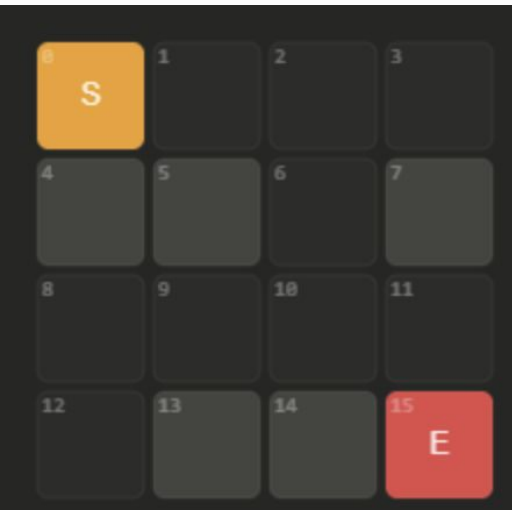
```
map();  
map(int l, int h);  
Case getCase(ivec2 vec) const;  
void set_casetype_in_grille(ivec2 vec, casetype type);  
int get_id(ivec2 vec) const;  
int get_largeur() const ;  
int get_hauteur() const ;  
int get_index_start() const ;  
int get_index_end() const ;  
int get_cell_size() const ;  
std::vector<Case>& getgrille() ;  
void generer_random_map();  
};
```

6 - Partie Logique

DFS – Depth-First Search



BFS, breadth first search



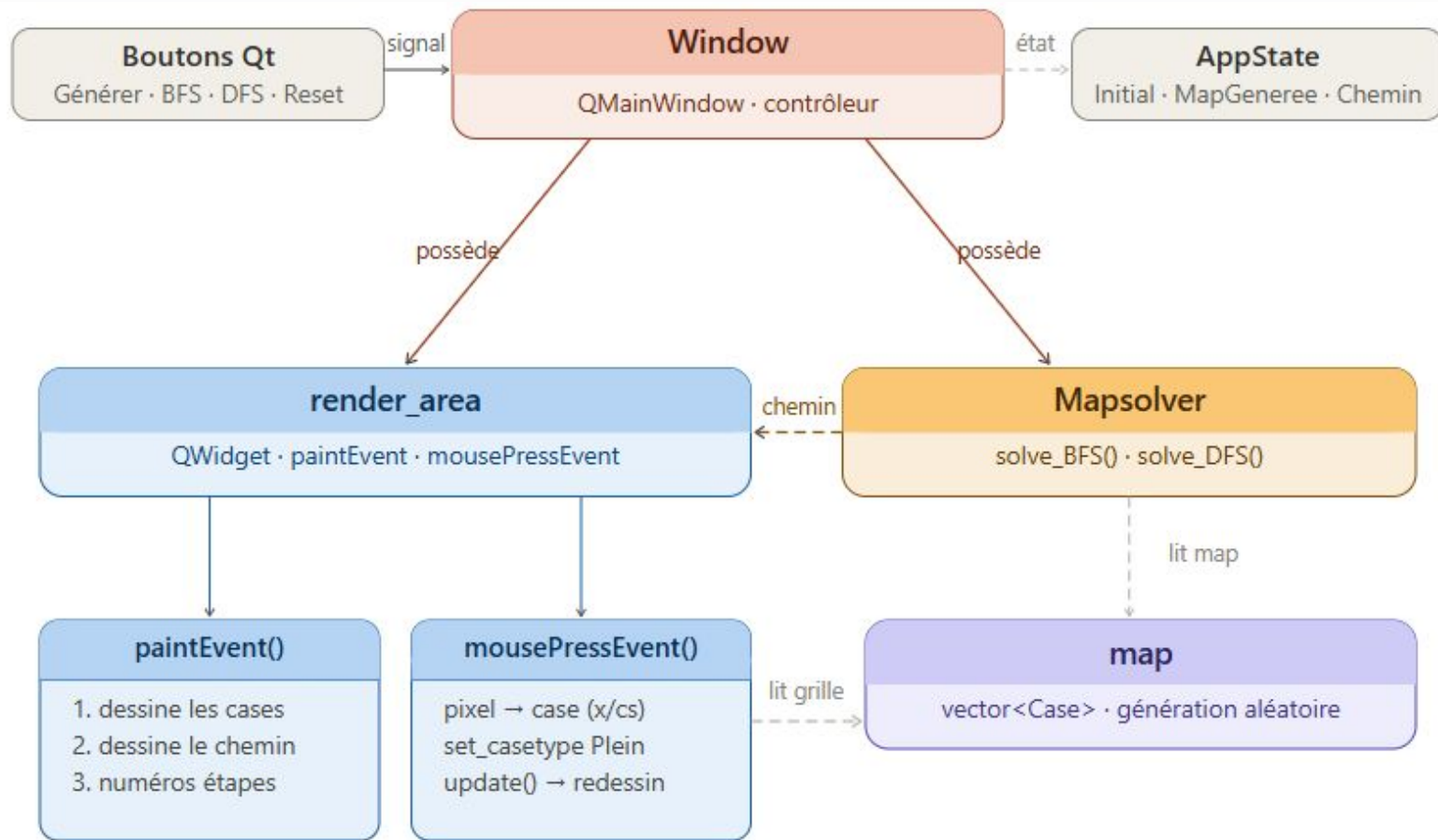
Comparaison BFS / DFS

QUAND UTILISER LEQUEL

?

Critère	BFS	DFS
Chemin optimal	Garanti	Non garanti
Structure	<code>std::queue</code>	<code>std::vector</code> (pile)
Labyrinthe simple	Rapide	Rapide
Chemin unique long	Peut être lent	Peut trouver vite
Implémentation	Itérative	Itérative

5 - Qt Affichage



7 - Limites et améliorations

Bilan et pistes d'amélioration

- **Limites du projet**

- Convention de nommage pas totalement cohérente (majuscules, style, uniformité)
- Organisation du projet perfectible
- Affichage limité au résultat final (pas de visualisation de l'exécution)
- Comparaison des algorithmes encore basique

- **Améliorations possibles**

- Ajouter différents types de cartes (ex : labyrinthes)
- Comparaison de performance selon le type de map

Merci de votre attention